

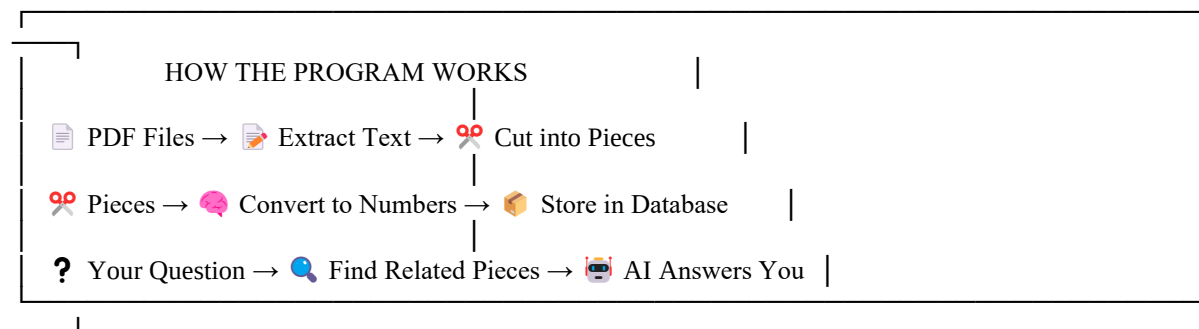
PDF Question-Answering System - Complete Code Explanation

What This Program Does (Simple Terms)

> ****Imagine you have a huge book and want to ask questions about it without reading the whole thing. This program reads PDF files, understands them, and answers your questions about them using AI!****

Visual Flow of the Program

...



Section 1: Importing Tools (Like Getting Your Equipment Ready)

```
```python
import os # Talks to your computer's operating system
import json # Handles data in a special format called JSON
import requests # Makes internet requests (like a web browser)
from dotenv import load_dotenv # Reads secret passwords from a file
from pypdf import PdfReader # Opens and reads PDF files
from langchain_text_splitters import RecursiveCharacterTextSplitter # Cuts text
from langchain_community.vectorstores import FAISS # Stores data
from langchain_community.embeddings import HuggingFaceEmbeddings # AI brain
from langchain_google_genai import GoogleGenerativeAIEmbeddings # Google AI
```
```

Simple Analogy Table

| Import | What It Does | Real World Analogy |
|-------------------------|-----------------------------|---------------------------------------|
| `os` | Talks to computer system | Like using File Explorer |
| `json` | Handles structured data | Like reading a form |
| `requests` | Sends internet requests | Like making a phone call |
| `dotenv` | Reads secret settings | Like a safe with passwords |
| `PdfReader` | Opens PDF files | Like opening a book |
| `TextSplitter` | Cuts text into pieces | Like cutting paper with scissors |
| `FAISS` | Stores & searches data fast | Like a super-organized filing cabinet |
| `HuggingFaceEmbeddings` | Converts text to numbers | Like translating to computer language |

Section 2: Loading Secret Keys (Your Password Manager)

```

```python
----- LOAD ENV -----
load_dotenv() # Step 1: Open the secret file (.env file)

Step 2: Get the secret API key from that file
OPENROUTER_API_KEY = os.getenv("OPENROUTER_API_KEY")

Step 3: If key is missing, stop and show error message
if not OPENROUTER_API_KEY:
 raise ValueError("OPENROUTER_API_KEY not found. Please set it in your .env file.")
```

```

What is an API Key?

...

| API KEY EXPLAINED | |
|---|---|
| Think of it like a LIBRARY CARD | |
|  | Library = OpenRouter AI Service |
|  | Library Card = API Key |
|  | You = The Program |
| Without the card → |  Cannot access books |
| With the card → |  Can access all AI features |
| .env file = Safe place to store your card number | |

...

What is a .env File?

...

This is what a .env file looks like (a simple text file)
Filename: .env

```
OPENROUTER_API_KEY=sk-or-v1-abc123yoursecretkeyhere
```

⚠ NEVER share this file with anyone!
⚠ It's like your bank account password

...

Section 3: The AI Brain (OpenRouter Connection)

```

```python
----- OPENROUTER LLM -----
class OpenRouterLLM:
 # LLM = Large Language Model (the AI that understands and generates text)

 def __init__(self, model="nvidia/nemotron-3-super-120b-a12b:free"):
 # __init__ runs automatically when we create this AI object
 # Think of it as "turning on" the AI

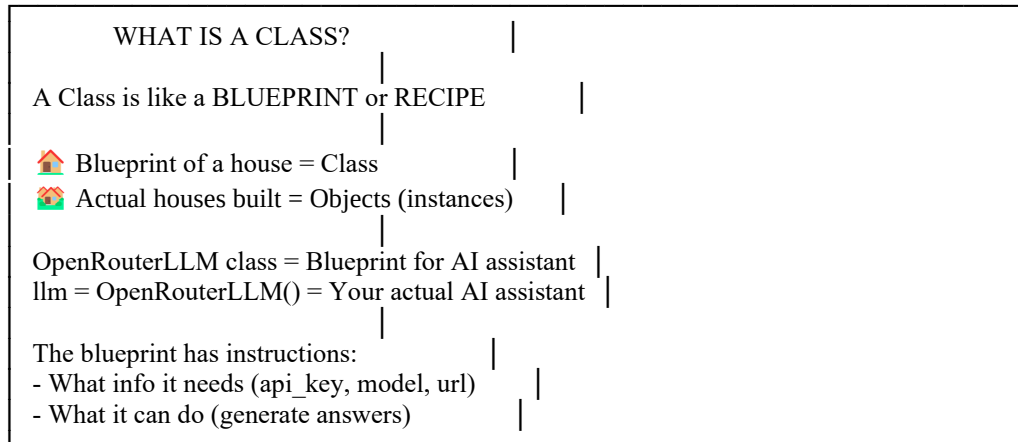
 self.api_key = OPENROUTER_API_KEY # Store our access key
 self.model = model # Which AI model to use
 self.url = "https://openrouter.ai/api/v1/chat/completions" # AI's address
```

```

```
...
```

Understanding Classes Simply

```
...
```



```
...
```

The Generate Function (Asking AI a Question)

```
```python
def generate(self, prompt):
 # This function SENDS your question to AI and GETS the answer
 # 'prompt' = the question/instruction we send to AI

 response = requests.post(
 # POST = sending data to a website (like submitting a form)

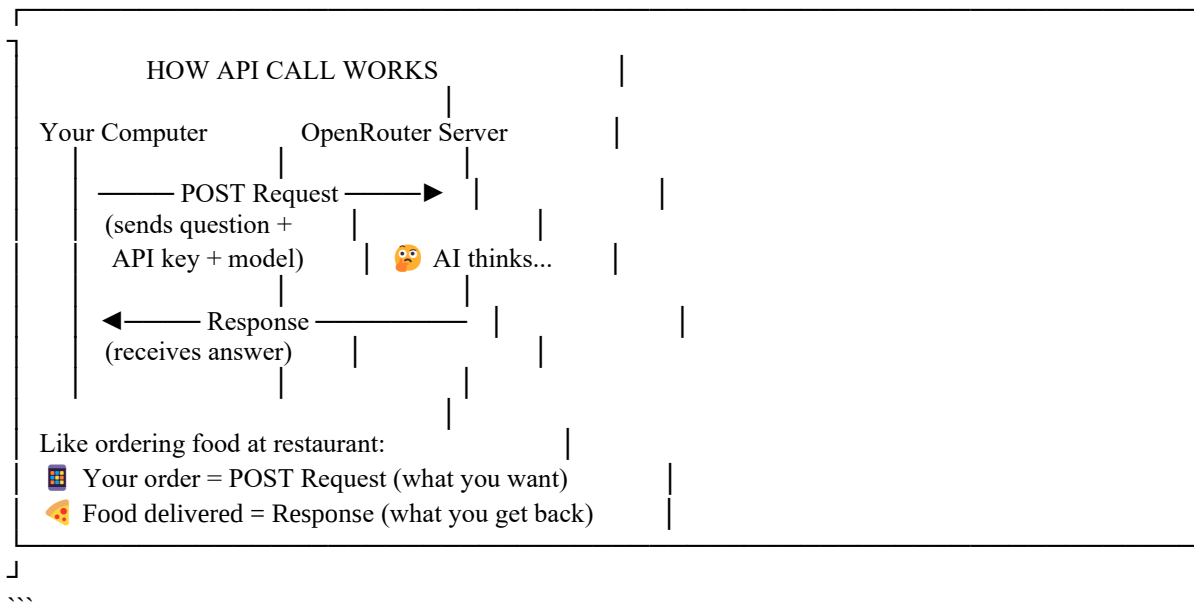
 url=self.url, # WHERE to send: OpenRouter's website address

 headers={
 # Headers = Extra information we send (like an envelope label)
 "Authorization": f"Bearer {self.api_key}", # Prove who we are
 "Content-Type": "application/json", # Tell format of data
 },

 data=json.dumps(
 # json.dumps = Convert Python data to JSON format
 # Like translating English to a language the server understands
 {
 "model": self.model, # Which AI model to use
 "messages": [
 {
 "role": "user", # We are the user
 "content": prompt # Our actual question
 }
],
 }
),
)
```
```

HTTP Request Visualization

```
...
```



```

...

'''python
# Get the response and convert it back to Python format
result = response.json()

# Check if we got a valid answer (choices = list of answers)
if "choices" not in result:
    # If something went wrong, stop and show the error
    raise Exception(f"OpenRouter API Error: {result}")

# Get the first answer from the list of choices
message = result["choices"][0]["message"]

# The message might come in different formats, handle both
if isinstance(message, dict):
    # If message is a dictionary (like {"content": "Hello"})
    return message.get("content", "") or ""

# If message is already plain text, convert to string
return str(message)
'''
  
```

Response Structure Explained

```

...

WHAT THE AI SENDS BACK

{
  "choices": [ ← List of possible answers
    {
      "message": { ← The actual message
        "role": "assistant", ← AI is speaking
        "content": "The answer is..." ← THE ANSWER
      }
    }
  ]
}

result["choices"][0]["message"]["content"]
  
```

= First choice → message → its content (answer) |

...

Section 4: Reading PDF Files

```
```python
----- PDF TEXT EXTRACTION -----
def get_pdf_text(pdf_docs):
 # pdf_docs = list of PDF files you want to read
 # Returns: all text from all PDFs combined

 text_parts = [] # Empty container to collect text pieces

 for pdf in pdf_docs:
 # Loop through each PDF file one by one

 try:
 # 'try' = attempt to do this (if it fails, go to 'except')

 pdf_reader = PdfReader(pdf) # Open the PDF file

 # Get the filename (or use "Document" if no name available)
 name = getattr(pdf, "name", "Document")

 for i, page in enumerate(pdf_reader.pages):
 # Loop through each page of the PDF
 # i = page number (0, 1, 2...)
 # page = actual page content

 content = page.extract_text() # Get text from this page

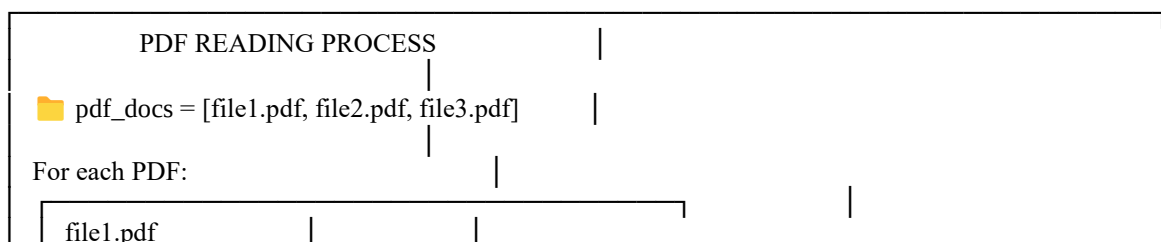
 if content: # Only add if page has text (not blank)
 text_parts.append(
 f"[{name} - Page {i + 1}]\n{content}"
 # Add label showing which file and page this is from
 # Example: "[Report.pdf - Page 1]\n The text here..."
)

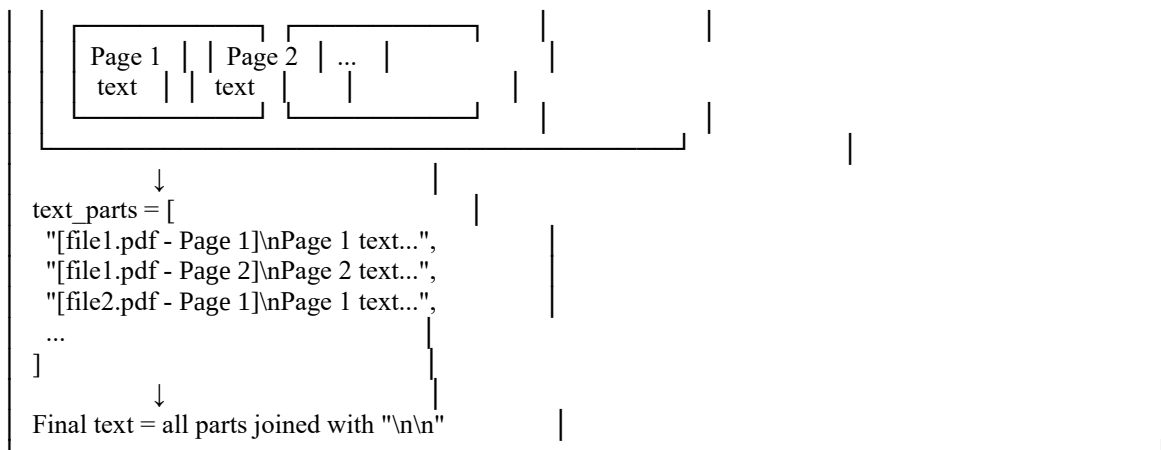
 except Exception as e:
 # If something goes wrong with a PDF, show error but continue
 print(f"Error: {e}")

 # Join all text parts with blank lines between them
 return "\n\n".join(text_parts)
```
```

PDF Reading Process Visualization

...





...

Section 5: Cutting Text into Manageable Pieces

```

```python
----- TEXT CHUNKING -----
def get_text_chunks(text):
 # Problem: AI has a limit on how much text it can read at once
 # Solution: Cut the big text into smaller "chunks"

 splitter = RecursiveCharacterTextSplitter(
 chunk_size=1000, # Each piece = maximum 1000 characters
 # (~150-200 words, about half a page)

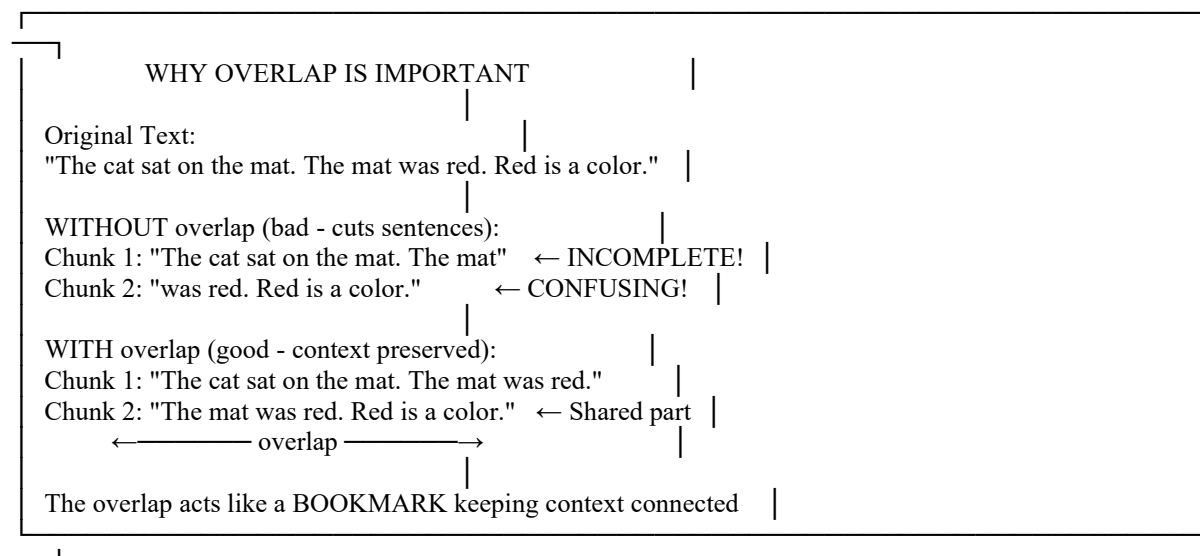
 chunk_overlap=200 # Each piece SHARES 200 characters with next piece
 # This prevents cutting important information in half
)

 return splitter.split_text(text) # Returns list of text pieces

```

## ### Chunk Overlap Visualization

...



...

...

Example with numbers:

---

Total text: 2500 characters

Chunk 1: Characters 1 → 1000 (1000 chars)

Chunk 2: Characters 801 → 1800 (1000 chars, shares 200 with Chunk 1)

Chunk 3: Characters 1601 → 2500 (1000 chars, shares 200 with Chunk 2)

---

...

---

## Section 6: Creating the Smart Search Database

```
```python
```

```
# ----- VECTOR STORE -----
```

```
def get_vector_store(chunks):
```

```
    # This converts text into numbers (vectors) so computer can  
    # understand and compare them mathematically
```

```
    embeddings = HuggingFaceEmbeddings(
```

```
        # HuggingFace = Website hosting AI models (like App Store for AI)
```

```
        model_name="sentence-transformers/all-MiniLM-L6-v2"
```

```
        # This specific AI model converts sentences to 384 numbers
```

```
        # It's free, fast, and accurate for finding similar text
```

```
    )
```

```
    # FAISS = Facebook AI Similarity Search
```

```
    # Creates a searchable database of all text chunks
```

```
    # Each chunk is stored as its number representation (vector)
```

```
    return FAISS.from_texts(chunks, embedding=embeddings)
```

```
```
```

### How Embeddings Work (Text to Numbers)

...

---

#### CONVERTING TEXT TO NUMBERS (EMBEDDINGS)

"The cat is sleeping" → [0.23, -0.45, 0.67, 0.12, ...]  
(384 numbers representing meaning)

"A kitten is napping" → [0.21, -0.43, 0.69, 0.11, ...]  
(similar numbers = similar meaning)

"Cars need gasoline" → [0.89, 0.34, -0.23, 0.78, ...]  
(different numbers = different topic)

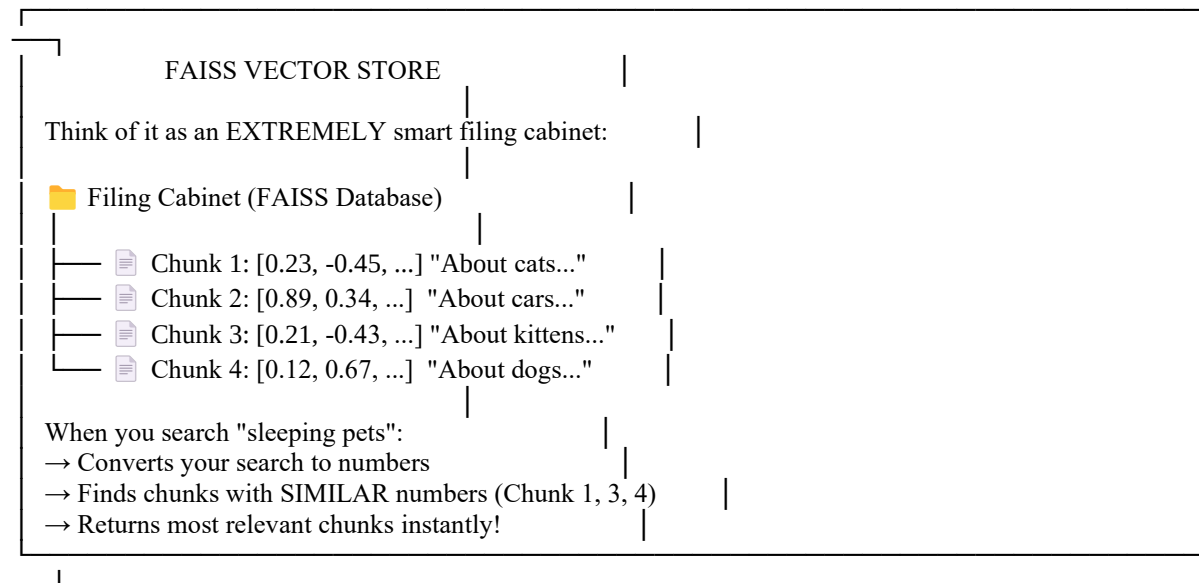
KEY INSIGHT: Similar MEANING = Similar NUMBERS  
This lets us find related text even with different words!

---

...

### ### FAISS Database Visualization

...



...

---

### ## Section 7: The Main Chat Function (Putting It All Together)

```
```python
```

```
# ----- RAG CHAT FUNCTION -----
```

```
def get_conversational_chain(vector_store):  
    # RAG = Retrieval Augmented Generation  
    # Fancy term for: "Find relevant info THEN generate answer"  
  
    llm = OpenRouterLLM()  # Create our AI assistant  
  
    def chat(input_dict):  
        # This inner function handles each question you ask  
        # Handle input whether it's a dictionary or plain text  
        if isinstance(input_dict, dict):  
            # If input is like {"question": "What is...?"}  
            question = input_dict.get("question", "")  
        else:  
            # If input is plain text like "What is...?"  
            question = input_dict  
  
        # STEP 1: Search for relevant information  
        docs = vector_store.similarity_search(question, k=3)  
        # similarity_search = Find most relevant text chunks  
        # k=3 = Return TOP 3 most relevant chunks  
  
        # STEP 2: Combine the relevant chunks into context  
        context = "\n\n".join([doc.page_content for doc in docs])  
        # doc.page_content = the actual text of each chunk  
  
        # STEP 3: Create a detailed prompt for the AI  
        prompt = f"""
```

You are a helpful AI assistant.

Answer the question based ONLY on the context below.
If the answer is not in the context, say "I don't know".

Context:
{context}

Question:
{question}

```
"""
    # The f-string inserts actual values of context and question
    # We tell AI to ONLY use the provided context
    # This prevents AI from making up (hallucinating) answers

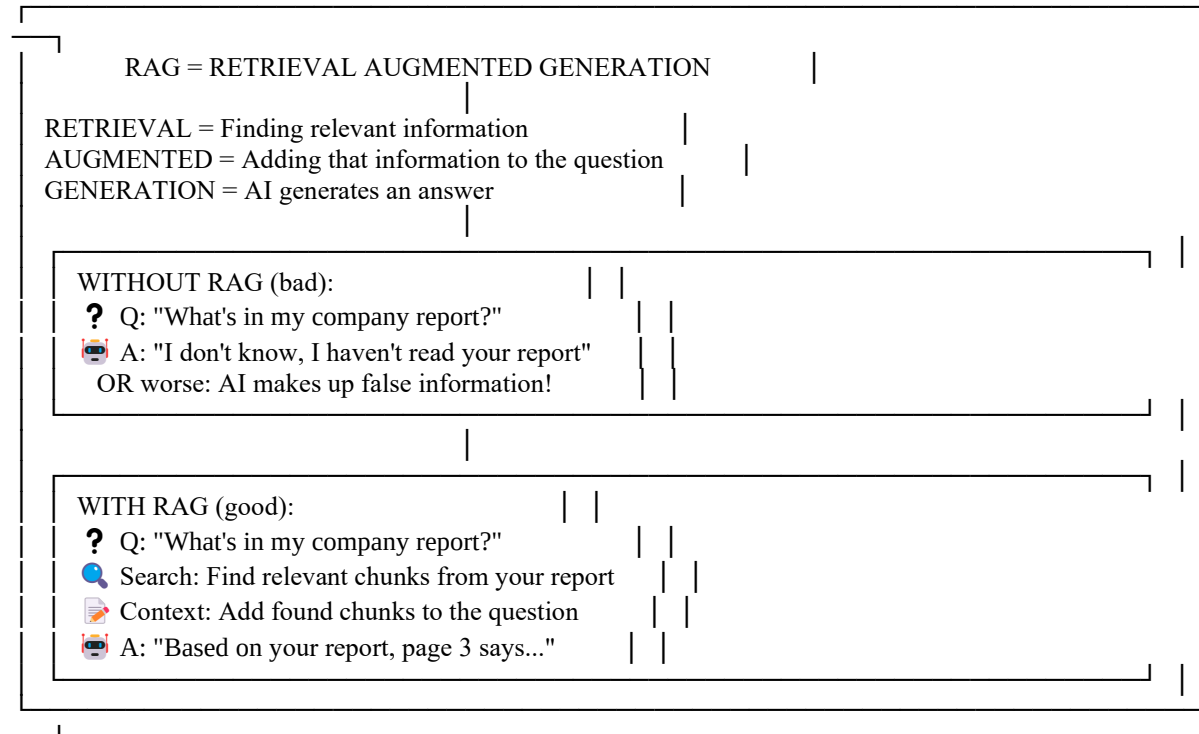
    # STEP 4: Get AI's answer
    answer = llm.generate(prompt)

    # STEP 5: Return both the answer and source documents
    return {
        "answer": answer,          # The AI's response
        "source_documents": docs  # Where info came from
    }

    return chat # Return the function so others can use it
"""
```

RAG Process Explained Simply

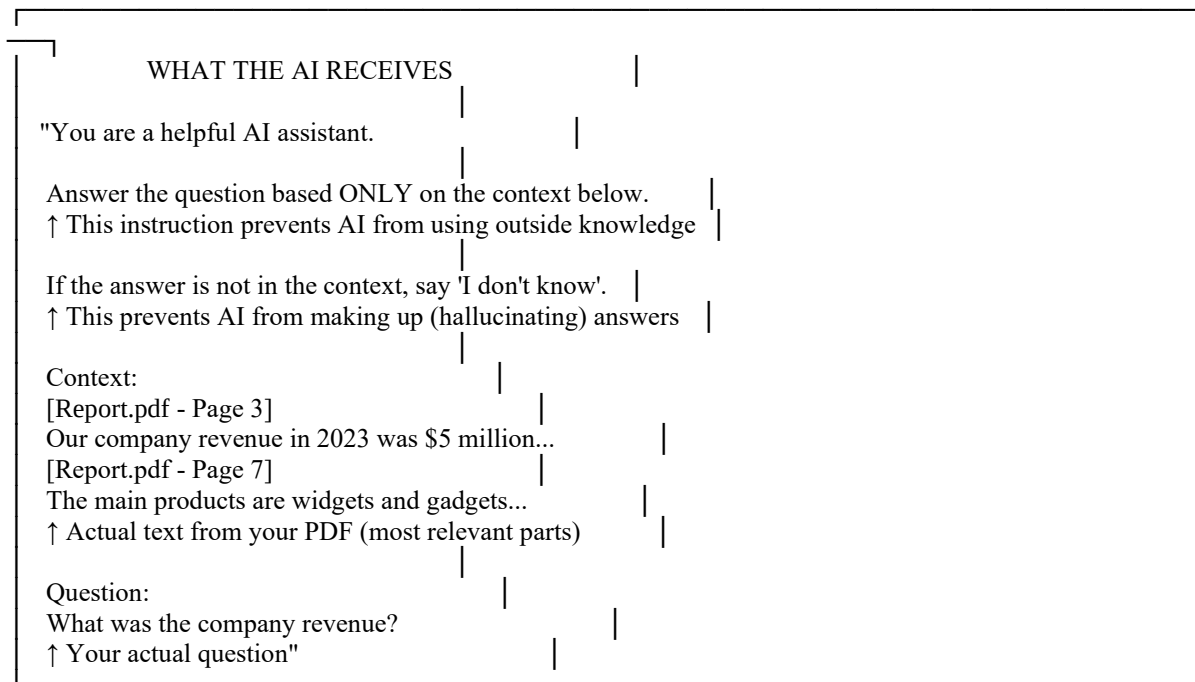
...



...

The Prompt Template Explained

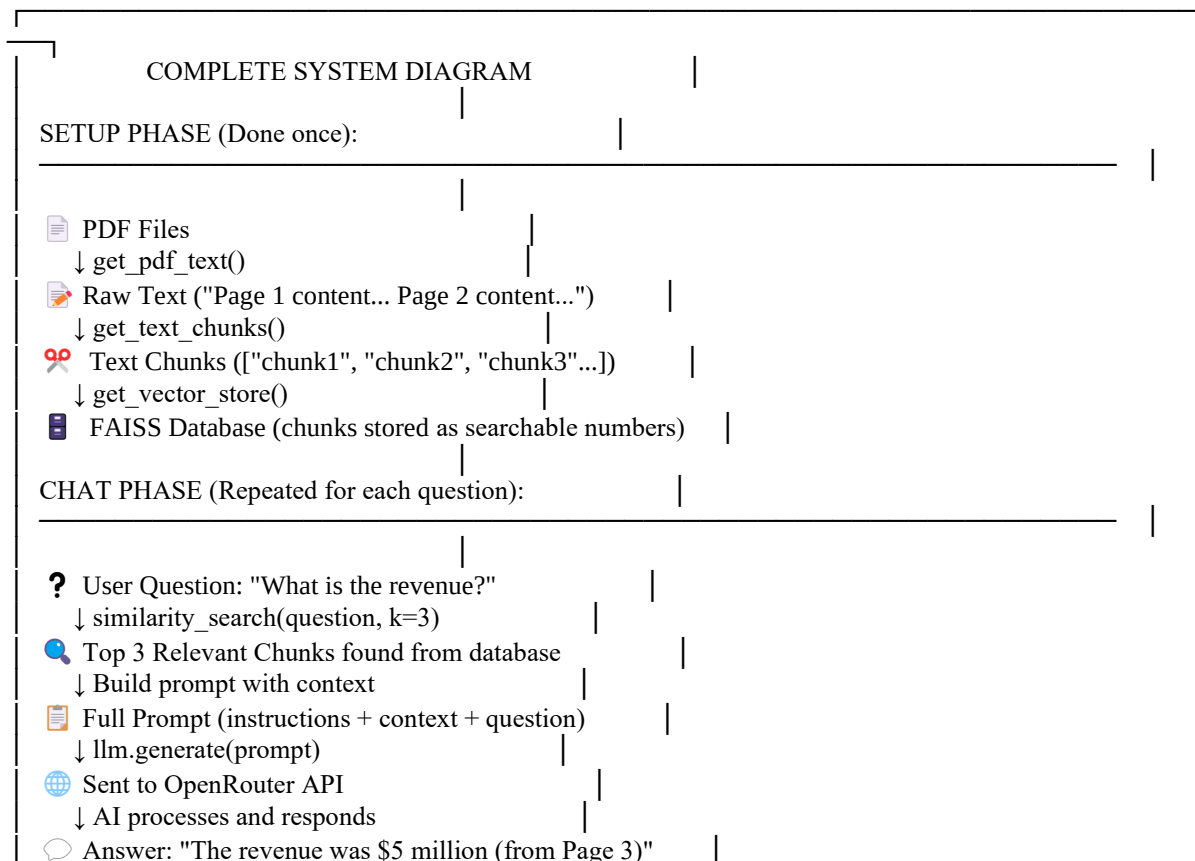
...



...

Complete System Flow

...



Key Concepts Summary Table

Concept	Technical Term	Simple Explanation	Real World Example
AI Model	LLM	A very smart text predictor	Like autocomplete but genius level
Number representation	Embedding/Vector	Text converted to numbers	Like converting words to GPS coordinates
Smart search database	Vector Store (FAISS)	Database that finds similar meanings	Like Google but for your documents
Text pieces	Chunks	Small pieces of text	Like index cards from a book
Sending data to AI	API Call	Asking the AI service online	Like ordering food via app
Secret passwords	API Keys	Permission to use a service	Like a membership card
Find relevant info	Similarity Search	Finding text with similar meaning	Like a librarian finding related books
Answer with sources	RAG	AI answers using your documents	Like a lawyer citing case files

Error Handling Explained

```
```python
Two types of error handling in this code:

Type 1: Try-Except (graceful failure)
try:
 pdf_reader = PdfReader(pdf) # Attempt risky operation
except Exception as e:
 print(f"Error: {e}") # If fails, show error and continue
Like: "Try to open the door, if locked, note it and try next door"

Type 2: Validation (stop if critical)
if not OPENROUTER_API_KEY:
 raise ValueError("Key not found!") # Stop everything
Like: "If you don't have your ticket, you cannot enter the show"
```
```

Summary: What Each Function Does

```
```
```

FUNCTION	PURPOSE
load_dotenv()	Load secret passwords from .env file
OpenRouterLLM()	Create AI assistant connection
.generate(prompt)	Send question, get AI answer
get_pdf_text()	Extract all text from PDF files
get_text_chunks()	Cut text into small searchable pieces
get_vector_store()	Build searchable number database
get_conversational_chain()	Create the question-answer function

```
| chat() | Search + build prompt + get answer |
```

---

```
> **Bottom Line:** This program lets you upload any PDF and ask questions about it. It reads the PDF, stores the content intelligently, and uses AI to give accurate answers based only on what's in your documents - no guessing!
```

## PDF QUESTION-ANSWERING SYSTEM

### Complete Code Explanation For Non-Technical Beginners

#### WHAT THIS PROGRAM DOES

Imagine you have a huge book and want to ask questions about it without reading the whole thing. This program reads PDF files, understands them, and answers your questions about them using AI!

#### PROGRAM FLOW

```
PDF Files --> Extract Text --> Cut into Pieces
Pieces --> Convert to Numbers --> Store in Database
Question --> Find Related Pieces --> AI Gives Answer
```

#### SECTION 1: IMPORTING TOOLS

Just like a chef gathers ingredients before cooking, this program gathers all the tools it needs:

```
import os = Talks to your computer
import json = Handles data formats
import requests = Makes internet requests
from dotenv = Reads secret passwords
from pypdf = Opens PDF files
FAISS = Smart search database
HuggingFace = AI brain for understanding text
```

#### SECTION 2: LOADING SECRET KEYS

What is an API Key?

- Like a LIBRARY CARD for AI services
- Without it: Cannot access the AI
- With it: Can use all AI features
- Stored in .env file (like a password safe)

⚠ SECURITY: NEVER share your .env file with anyone!  
It's like your bank password.

### SECTION 3: THE AI BRAIN (OpenRouter Connection)

class OpenRouterLLM:

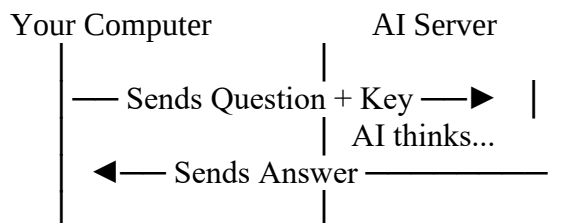
This is a BLUEPRINT for creating an AI assistant.

Like a house blueprint:

- Blueprint = Class (instructions)
- Actual house = Object (working AI)

The generate() function sends questions to AI and gets answers.

#### HOW API CALL WORKS:



Like ordering food at a restaurant:

🍽 Your order = The request

🍴 Food delivered = The response

### SECTION 4: READING PDF FILES

The program opens each PDF file and extracts text from EVERY page.  
It labels each piece with filename and page number.

Example output:

[Report.pdf - Page 1]

Company annual report for 2023...

[Report.pdf - Page 2]

Revenue increased by 25%...

### SECTION 5: CUTTING TEXT INTO PIECES

AI has limits on how much it can read at once.

Solution: Cut text into "chunks" (pieces).

Chunk size: 1000 characters max

Overlap: 200 characters shared between chunks

Why overlap? It prevents cutting sentences in half!

WITHOUT overlap (bad):

Chunk 1: "The cat sat on the mat. The mat" ← INCOMPLETE

Chunk 2: "was red. Red is a color." ← CONFUSING

WITH overlap (good):

Chunk 1: "The cat sat on the mat. The mat was red."

Chunk 2: "The mat was red. Red is a color." ← Context preserved!

## SECTION 6: CONVERTING TEXT TO NUMBERS (EMBEDDINGS)

Computers understand numbers, not words. This program converts every text piece into 384 numbers.

Amazing fact: Similar meanings = Similar numbers!

"The cat is sleeping" → [0.23, -0.45, 0.67, 0.12, ...]

"A kitten is napping" → [0.21, -0.43, 0.69, 0.11, ...] (SIMILAR!)

"Cars need gasoline" → [0.89, 0.34, -0.23, 0.78, ...] (DIFFERENT!)

FAISS DATABASE: An extremely smart filing cabinet that finds text by MEANING, not just exact words.

## SECTION 7: RAG - THE SMART ANSWER SYSTEM

RAG = Retrieval Augmented Generation

Step 1: RETRIEVAL = Find relevant info from your PDFs

Step 2: AUGMENTED = Add that info to your question

Step 3: GENERATION = AI generates an answer

WITHOUT RAG (bad):

Q: "What's in my report?"

A: "I don't have your report" or makes up false info!

WITH RAG (good):

Q: "What's in my report?"

A: "Based on your report page 3: Revenue was \$5 million..."

## SECTION 8: COMPLETE SYSTEM SUMMARY

SETUP PHASE (Done once):

PDF Files → Extract Text → Cut into Chunks → Convert to Numbers  
→ Store in Database

CHAT PHASE (Each question):

Your Question → Search Database → Find Relevant Pieces  
→ Build Prompt → Send to AI → Get Answer

## KEY CONCEPTS SUMMARY

Concept	Simple Explanation	Real-World Analogy
AI Model	Smart text predictor	Genius autocomplete
Embedding	Text as numbers	Words to GPS coordinates
FAISS	Search by meaning	Google for your docs
Chunks	Small text pieces	Index cards from book
Similarity	Find related meaning	Librarian finds books
RAG	AI uses your documents	Lawyer citing case files
API Key	Permission to use service	Library membership card

## ERROR HANDLING

The program has two types of error handling:

1. Try-Except: "Try this, if it fails, note it and continue"  
Like: Try a door, if locked, try the next door
2. Raise Error: "This is critical, stop everything if missing"  
Like: No ticket = Cannot enter the show

## BOTTOM LINE

Upload any PDF, ask questions, get accurate AI answers  
based ONLY on what's in your documents. No guessing!

=====